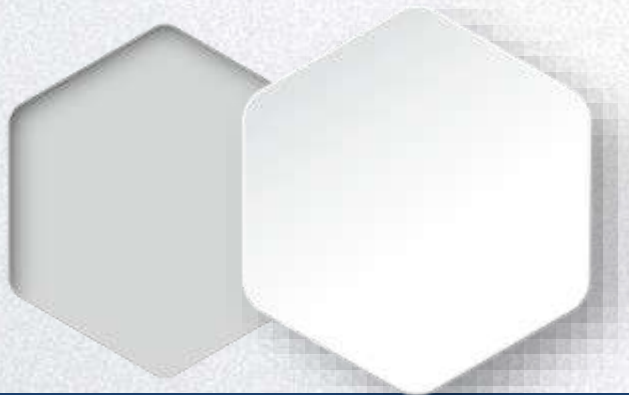




《人工智能与Python程序设计》——初识Python



人工智能与Python程序设计 教研组



提纲



初识Python

- ☐ 计算机程序设计语言
- ☐ Python程序实例
-
-



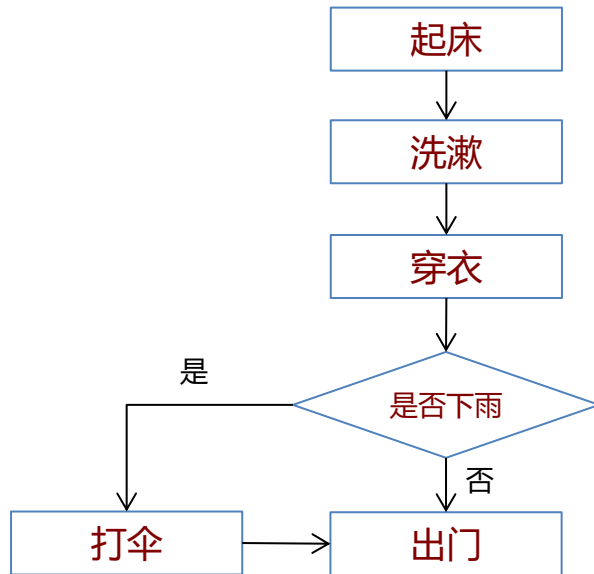
什么是程序？

- **程序 (Program)**

- 流程、议程、行程、...
- 为了完成某项任务，解决某个问题所需要执行的一系列步骤

- **计算机程序**

- 为了完成某项任务，解决某个问题由**计算机**执行的一系列指令（步骤）



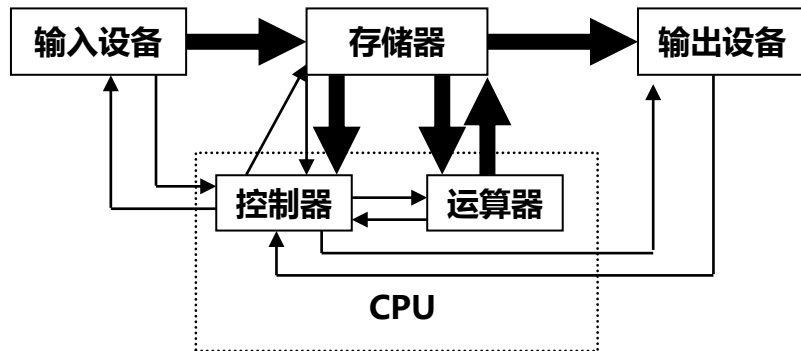
什么是计算机？

- 计算机 (Computer)
 - 能够按照程序自动运行的机器
- 组成
 - 硬件 (Hardware)
 - 计算机的躯壳
 - 软件 (Software)
 - 计算机程序
 - 计算机的灵魂



计算机的基本组成

- 冯•诺依曼体系结构



- 计算机之父：冯•诺依曼
 - 计算机应该按照程序顺序执行
 - 采用二进制作为计算机的数制基础

机器执行的程序（最初形式）



10111000
00000001
00000000
00000101
00000001
00000000

- Compute 1+1



汇编语言 (Assembly Language)



MOV AX, 1

10111000
00000001
00000000

ADD AX, 1

00000101
00000001
00000000



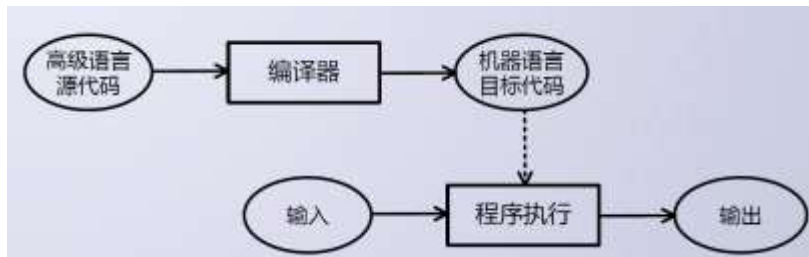
高级语言 (High-level Language)

- 如: Python 语言

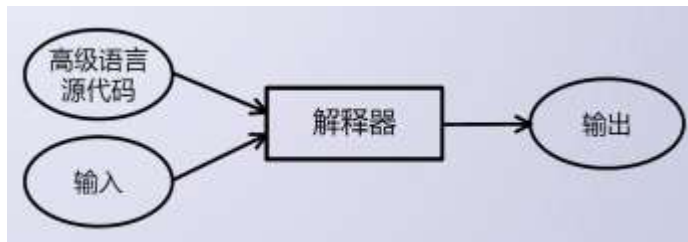
```
print(1 + 1)
```

Recap: Python是解析型高级语言

- 编译型语言 (C/C++等)



- 解释型语言 (BASIC、Python等)





提纲



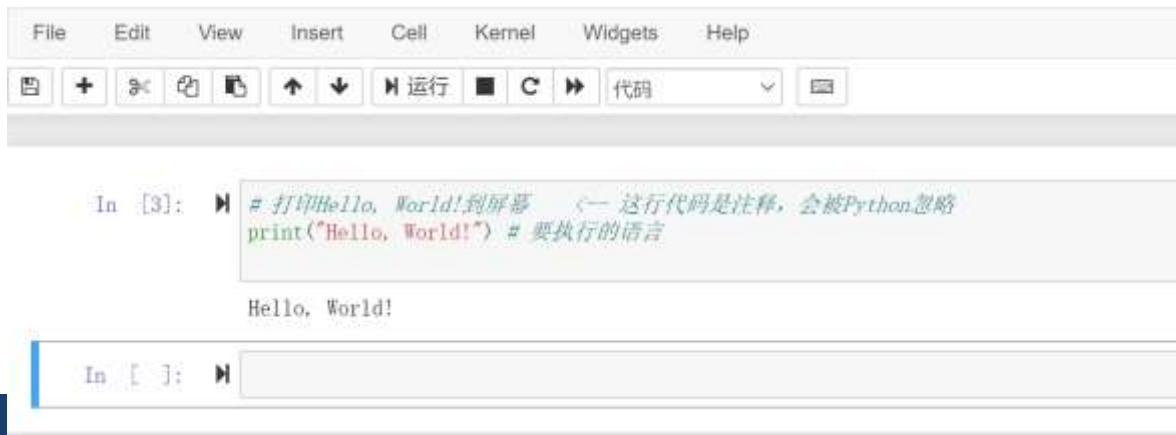
初识Python

- ☐ 计算机程序设计语言
- ☐ Python程序实例
-
-



你的第一个Python程序: "Hello World"

- 它是编程之神的传统咒语，可以帮助你开始这段感情.....
 - 注释：每一行 # 后面都是注释
 - Python命令：print
 - 输入：字符串 "Hello, World"
 - 输出： "Hello, World"





更多Python的例子

- `print(2 * 3)`
 - 打印数字2*3的结果
 - 2和3是常量，不可以改变
- a、b、c是变量，=为赋值符号
 - `a = 2`: a赋值2
 - `b = 3`: b赋值3
 - `c = a + b`: 先算a+b, 结果赋值c
 - `c = a * b`: 先算a*b, 结果赋值c
覆盖了原来a+b的值

```
In [4]: ▶ print(2*3)
```

6

```
In [6]: ▶ a = 2  
b = 3  
c = a + b  
print(c)  
c = a * b  
print(c)
```

5

6



温度转换程序实例：问题描述

- 温度刻画存在不同体系
 - **摄氏度**：以1标准大气压下水的结冰点为0度，沸点为100度，将温度进行等分刻画。
 - **华氏度**：以1标准大气压下水的结冰点为32度，沸点为212度，将温度进行等分刻画。
- 其它温度刻画体系
 - 开氏温标：描述客观世界真实的温度.....



温度转换程序实例：问题分析

- **问题：**如何利用Python程序进行**摄氏度和华氏度之间的转换**
 - 步骤1：分析问题的计算部分：采用公式转换方式解决计算问题
 - 步骤2：确定功能
 - 输入：华氏或者摄氏温度值、温度标识
 - 处理：温度转化算法
 - 输出：华氏或者摄氏温度值、温度标识
 - 步骤3：设计算法，转换公式如下：
 - $C = (F - 32) / 1.8$
 - $F = C * 1.8 + 32$

F表示华氏度，82F表示华氏82度

C表示摄氏度，28C表示摄氏28度



温度转换程序实例：编写程序

- **问题：**如何利用Python程序进行**摄氏度**和**华氏度**之间的转换
 - 步骤3：设计算法，转换公式如下：
 - $C = (F - 32) / 1.8$
 - $F = C * 1.8 + 32$
 - 步骤4：编写程序

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```



温度转换程序实例：程序运行

- 运行方法1（本门课的主要方法）
 - 在Jupyter Notebook中输入源程序
 - 点击“运行”按钮
- 适合实验和教学

The image shows a Jupyter Notebook window titled "Python-01" with a status bar indicating "最后检查: 11 小时前 (未保存改变)". The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar contains icons for file operations, a "运行" (Run) button, and a dropdown menu currently set to "代码" (Code). The code cell, labeled "In [7]:", contains the following Python code:

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值:")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

Below the code, the output is displayed:

```
请输入带有符号的温度值: 50F
转换后的温度是10.00C
```

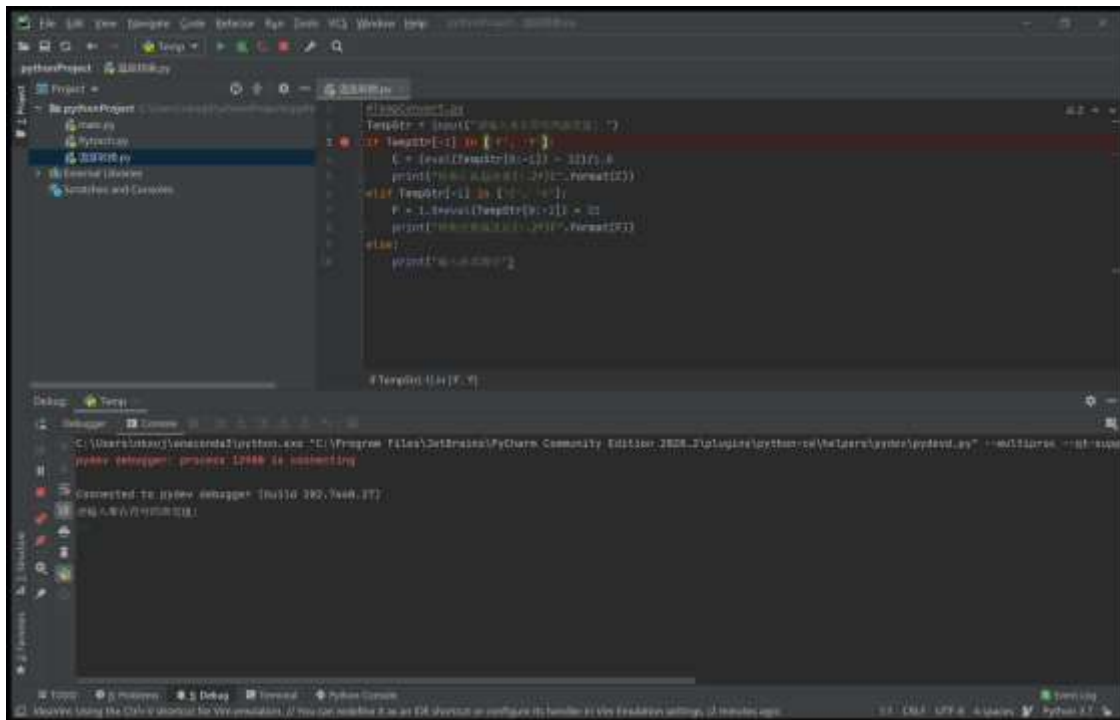


温度转换程序实例：程序运行

- 运行方法2：通过IDE进行运行和Debug

- 编辑程序
- 设置断点
- 单步运行
- 查看变量值

- 适合编写规模较大的项目和工程





温度转换程序实例：程序运行

- 运行方法3：通过Python命令运行
 - 启动Anaconda Prompt
 - 确认.py文件的地址
 - 执行命令行 `python <.py文件>`

```
1 #tempConvert.py
2 TempStr = input("请输入带有符号的温度值:")
3 if TempStr[-1] in ['F', 'f']:
4     C = (eval(TempStr[0:-1]) - 32)/1.8
5     print("转换后的温度是{:.2f}C".format(C))
6 elif TempStr[-1] in ['C', 'c']:
7     F = 1.8*eval(TempStr[0:-1]) + 32
8     print("转换后的温度是{:.2f}F".format(F))
9 else:
10    print("输入格式错误")
```

```
(base) C:\Users\nkxu>python C:\Users\nkxu\PycharmProjects\pythonProject\温度转换.py
请输入带有符号的温度值: 35
输入格式错误

(base) C:\Users\nkxu>python C:\Users\nkxu\PycharmProjects\pythonProject\温度转换.py
请输入带有符号的温度值: 35F
转换后的温度是1.67C

(base) C:\Users\nkxu>
```



温度转换程序实例：程序运行

- 运行方法4：在Python环境中输入和运行
 - 启动Anaconda Prompt
 - 执行Python命令，启动Python环境
 - 一遍输入代码一边执行

- 适合快速验证Python语句的功能

```
(base) C:\Users\nkxu>python
Python 3.7.6 (default, Jan 8 2020, 20:23:39) [MSC v.1916 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type 'help', 'copyright', 'credits' or 'license' for more information
Failed calling sys._interactivebook_
Traceback (most recent call last):
  File "C:\Users\nkxu\anaconda3\lib\site.py", line 439, in register_readline:
    readline.read_history_file(history)
  File "C:\Users\nkxu\anaconda3\lib\site-packages\pyreadline\rlmain.py", line 165, in read_history_file:
    self.mode.read_history_file(filename)
  File "C:\Users\nkxu\anaconda3\lib\site-packages\pyreadline\linceditor\history.py", line 82, in read_history_file:
    for line in open(filename, 'r'):
UnicodeDecodeError: 'gbk' codec can't decode byte 0xaf in position 47: illegal multibyte sequence
>>> TempStr = input("请输入带有符号的温度值:")
请输入带有符号的温度值: 35F
>>> if TempStr[-1] in ['F', 'f']:
...     C = (eval(TempStr[0:-1]) - 32)/1.8
...     print("转换后的温度是{:.2f}C".format(C))
...
转换后的温度是1.67C
>>>
```



程序元素分析：程序的格式框架

- Python语言采用严格的“缩进”来表明程序的格式框架。
 - 缩进指每一行代码开始前的空白区域，建议用Tab键表示缩进
- 1个缩进=4个空格
- 用以在Python中标明代码的层次关系
- 缩进是Python语言中表明程序框架的唯一手段
 - 其它语言如C和Java用大括号{ }表示
- 缩进可以有多层，以表示更复杂的逻辑关系

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

```
DARTS = 1000
hits = 0.0
clock()
for i in range(1, DARTS):
    x, y = random(), random()
    dist = sqrt(x ** 2 + y ** 2)
    if dist <= 1.0:
        hits = hits + 1
pi = 4 * (hits/DARTS)
print("Pi的值是{:.2f}".format(pi))
```

程序元素分析：程序的格式框架

- Python的注释以#号开头，#号之后一直到一行末尾的所有字符，都是注释的一部分，Python解释器将忽略它们，不予执行。
- 多行注释以'''开头和结尾（做大段地说明）
- 注释有助于提升程序的可读性，有百利而无一害
 - 帮助他人读懂你的程序提出建议改正错误
 - 帮助以后的你读懂你自己的程序以便复用
 - 帮助你的同事使用你的程序扩大影响力
 - 唯一的缺点：懒、嫌麻烦

```
# 打印Hello, World!到屏幕    <— 这行代码是注释，会被Python忽略
print("Hello, World!") # 要执行的语言
```

Hello, World!



```
#TempConvert.py
TempStr = input("请输入带有符号的温度值:")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```



程序元素分析：命名

- 两类标识符
 - 常量：程序中值不发生改变的元素
 - 变量：程序中值发生改变或者可以发生改变的元素
- Python语言允许采用**大写字母、小写字母、数字、下划线和汉字**等字符及其组合给变量命名
 - 但名字的首字符不能是数字
 - 中间不能出现空格
 - 长度没有限制
 - 标识符对大小写敏感，python和Python是两个不同的名字
- 示例程序中的标识符
 - TempStr
 - C
 - F

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```



程序元素分析：保留字

- 保留字，也称为关键字，指被编程语言内部定义并保留使用的标识符。
 - 程序员编写程序**不能**定义与保留字相同的标识符。
 - 每种程序设计语言都有一套保留字，保留字一般用来构成程序整体框架、表达关键值和具有结构性的复杂语义等。
 - Python 3.x一共有33个保留字
 - 掌握一门编程语言首先要熟记其所对应的保留字
- 示例程序中的保留字
 - if、elif、else、in

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值:")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

and	elif	import	raise
as	else	in	return
assert	except	is	try
break	finally	lambda	while
class	for	nonlocal	with
continue	from	not	yield
def	global	or	True
del	if	pass	False
			None



程序元素分析：字符串

- Python语言中，字符串是用两个双引号“字符串”或者单引号‘字符串’括起来的一个或多个字符。
- Python字符串的两种序号体系
 - 从左到右，从0开始逐位递增
 - 从右到左，从-1开始，逐位递减 🤔
- 示例中的字符串处理
 - TempStr[-1]：取字符串最后一个字符
 - Temp[0:-1]：取字符串从第1个到倒数第二个 🤔

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```





程序元素分析：赋值语句

- Python语言中，等于号=表示“赋值”
 - 将等号右侧的值计算后将结果值赋给左侧变量，包含等号(=)的语句称为“赋值语句”
 - 切记：这不是数学中的相等!!!!

```
TempStr = input("请输入带有符号的温度值：")
```

表示：把input（用户输入的字符串）返回的值放到TempStr这个字符串变量中

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

```
➤ a = 5
print(a)
print(a == 5)
```

```
5
True
```



程序元素分析：同步赋值

- 同步赋值语句：同时给多个变量赋值
 - $\langle \text{变量}1 \rangle, \dots, \langle \text{变量}N \rangle = \langle \text{表达式}1 \rangle, \dots, \langle \text{表达式}N \rangle$
 - $A, B = 12, 13$ 等价于 $A = 12 \quad B = 13$
 - 好处：省代码

- 举例：交换A和B的值

```
In [10]: A = 10
          B = 12

          print(A, B)
          C = A
          A = B
          B = C
          print(A, B)

10 12
12 10
```

```
In [11]: print(A, B)
          A, B = B, A
          print(A, B)

12 10
10 12
```



程序元素分析： input()函数

- input()是一个系统内置的函数，获取并返回用户在命令行中的输入字符串
 - 注意：input不是保留字
 - 获得用户输入之前，input()函数可以包含一些提示性文字
 - `<变量> = input(<提示性文字>)`

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```

请输入带有符号的温度值：



程序元素分析： eval()函数

- `eval(<字符串>)` 是Python语言中一个很特殊的函数
 - 以Python表达式的方式解析并执行字符串，将返回结果输出
 - 下面两句语句实现了相同的功能

```
TempStr = "50"

C = (eval(TempStr) - 32)/1.8
print(C)

C1 = (int(TempStr) - 32)/1.8
print(C1)

10.0
10.0
```

```
TempStr = "50"
C = (TempStr - 32)/1.8
```

```
TypeError                                 Traceback (most recent call last)
<ipython-input-17-4a32ae7889e2> in <module>
      1 TempStr = "50"
----> 2 C = (TempStr - 32)/1.8

TypeError: unsupported operand type(s) for -: 'str' and 'int'
```



程序元素分析：输出函数print()

- print()函数用来输出字符信息，或以字符形式输出变量。
 - 可以输出各种类型变量的值
 - 可以通过%来选择要输出的变量

```
num1 = input("The first number is")
num2 = input("The second number is")
avg_num = (float(num1) + float(num2)) / 2
print("The average number is %f" % avg_num)
```

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值：")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```



程序元素分析：流程控制语句

- if: 如果满足某个, 则执行其控制下 (缩进的) 程序部分 (跳过else和elif部分)
- else: 与if相对应, 如果不满足, 则执行其控制下的程序
- elif: else if的缩写
 - 如果第一个if不满足条件, 抢在后面的else之前再判断一次 (截胡)
 - 如果满足条件, 执行其控制的程序 (跳过else部分)
 - elif可以有0-N个

```
#TempConvert.py
TempStr = input("请输入带有符号的温度值:")
if TempStr[-1] in ['F', 'f']:
    C = (eval(TempStr[0:-1]) - 32)/1.8
    print("转换后的温度是{:.2f}C".format(C))
elif TempStr[-1] in ['C', 'c']:
    F = 1.8*eval(TempStr[0:-1]) + 32
    print("转换后的温度是{:.2f}F".format(F))
else:
    print("输入格式错误")
```



Python画图程序实例：问题描述

- 用Python画一条蟒蛇



- 问题1: 计算机绘图是什么原理?
 - 一段程序为何能够产生窗体？为何能在窗体上绘制图形？
- 问题2: Python蟒蛇绘制从哪里开始？
 - 如何绘制一条线？如何绘制一个弧形？如何绘制一个蟒蛇？



示例程序：在Jupyter Notebook中运行

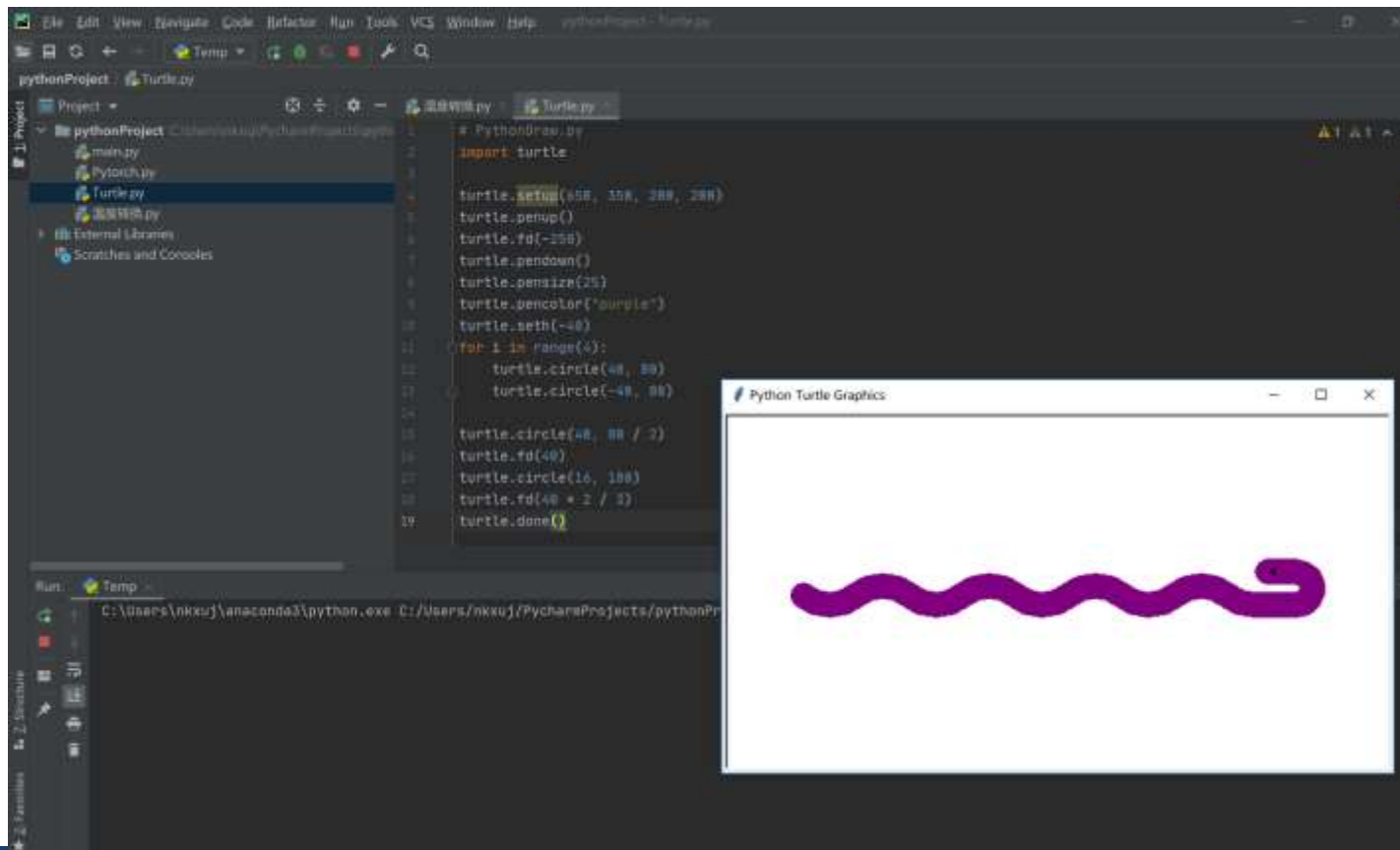
```
In [*]: ▶ #PythonDraw.py
import turtle
turtle.setup(650 , 350, 200, 200)
turtle.penup()
turtle.fd(-250)
turtle.pendown()
turtle.pensize(25)
turtle.pencolor("purple")
turtle.seth(-40)
for i in range(4):
    turtle.circle(40,80)
    turtle.circle(-40, 80)

turtle.circle(40, 80/2)
turtle.fd(40)
turtle.circle(16,180)
turtle.fd(40 * 2/3)
turtle.done()
```

A screenshot of a Jupyter Notebook interface. On the left, a code cell is shown with the following Python code: `In [*]: ▶ #PythonDraw.py`, `import turtle`, `turtle.setup(650 , 350, 200, 200)`, `turtle.penup()`, `turtle.fd(-250)`, `turtle.pendown()`, `turtle.pensize(25)`, `turtle.pencolor("purple")`, `turtle.seth(-40)`, `for i in range(4):`, `turtle.circle(40,80)`, `turtle.circle(-40, 80)`, `turtle.circle(40, 80/2)`, `turtle.fd(40)`, `turtle.circle(16,180)`, `turtle.fd(40 * 2/3)`, `turtle.done()`. On the right, a window titled "Python Turtle Graphics" displays the result of the code: a thick purple wavy line with a small circle at the end, resembling a stylized snake or a decorative flourish.



示例程序：在PyCharm中运行





Python画图程序解析：库引用

- import turtle
 - import 为保留字
 - 引入会图库turtle
 - 更加简洁的用法

import turtle as t

- 引入函数

from turtle import <函数名>
from turtle import *

```
import turtle

turtle.setup(650 , 350, 200, 200)
turtle.penup()
turtle.fd(-250)
turtle.pendown()
turtle.pensize(25)
turtle.pencolor("purple")
turtle.seth(-40)
for i in range(4):
    turtle.circle(40,80)
    turtle.circle(-40, 80)

turtle.circle(40, 80/2)
turtle.fd(40)
turtle.circle(16,180)
turtle.fd(40 * 2/3)
turtle.done()
```

```
import turtle as t

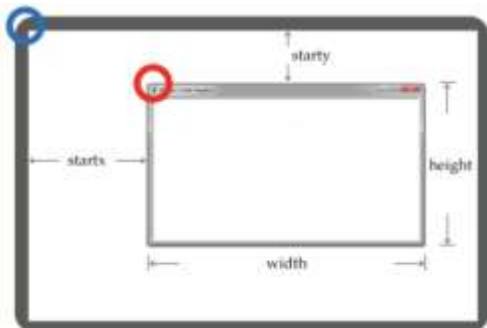
t.setup(650 , 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40,80)
    t.circle(-40, 80)

t.circle(40, 80/2)
t.fd(40)
t.circle(16,180)
t.fd(40 * 2/3)
t.done()
```



Python画图程序解析：构建画板

- `t.setup(width=_CFG["width"], height=_CFG["height"], startx=_CFG["leftright"], starty=_CFG["topbottom"])`
 - 设置主窗口的大小和位置。
 - `width` -- 默认为屏幕的 50%
 - `height` -- 默认为屏幕的 75%
 - `startx` -- 表示初始位置距离屏幕左边缘多少像素
 - `starty` -- 表示初始位置距离屏幕上边缘多少像素



```
import turtle as t

t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40, 80)
    t.circle(-40, 80)

t.circle(40, 80/2)
t.fd(40)
t.circle(16, 180)
t.fd(40 * 2/3)
t.done()
```

https://blog.51cto.com/u_16213669/10705300



Python画图程序解析：画笔控制

- `t.penup()`: 提笔, 不留笔迹
- `t.pendown()`: 下笔, 开始画
- `t.pensize(width)`: 画笔宽度
- `t.pencolor( )`: 画笔颜色
 - 颜色值
 - RGB **`t.pencolor(0.63, 0.13, 0.94)`**

```
import turtle as t

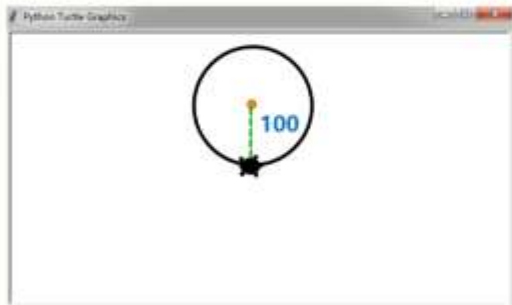
t.setup(650 , 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40,80)
    t.circle(-40, 80)

t.circle(40, 80/2)
t.fd(40)
t.circle(16,180)
t.fd(40 * 2/3)
t.done()
```

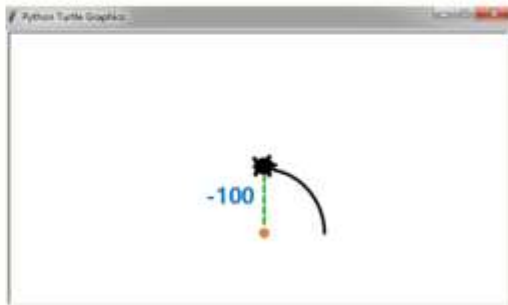
Python画图程序解析：画笔运动

- `t.fd(d)`
 - 海龟前进 *distance* 指定的距离，方向为海龟的朝向。
- `t.circle(radius, extent=None)`
 - 绘制一个 *radius* 指定半径的圆，圆心在海龟左边 *radius* 个单位；*extent* 为一个夹角，用来决定绘制圆的一部分

```
turtle.circle(100)
```



```
turtle.circle(-100,90)
```



```
import turtle as t

t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40,80)
    t.circle(-40, 80)

t.circle(40, 80/2)
t.fd(40)
t.circle(16,180)
t.fd(40 * 2/3)
t.done()
```

Python画图程序解析：方向控制

- `turtle.seth(to_angle)`
 - 设置海龟的朝向为 `to_angle`
- `turtle.left (angle )`
 - 海龟向左转
- `turtle.right (angle)`
 - 海龟向右转

`turtle.seth(45)`

`turtle.seth(-135)`



```
import turtle as t

t.setup(650 , 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40,80)
    t.circle(-40, 80)

t.circle(40, 80/2)
t.fd(40)
t.circle(16,180)
t.fd(40 * 2/3)
t.done()
```



Python画图程序解析：循环语句与Range()函数

for<变量> in range(<次数>):
<被循环执行的语句>

- range(N): 产生整数序列 [0, 1, 2, ..., N-1]
- <变量>表示每次循环的计数，0到<次数>-1

```
>>> for i in range(5):  
    print(i)
```

```
0  
1  
2  
3  
4  
Hello: 0  
Hello: 1  
Hello: 2  
Hello: 3  
Hello: 4
```

```
>>> for i in range(5):  
    print("Hello:",i)
```

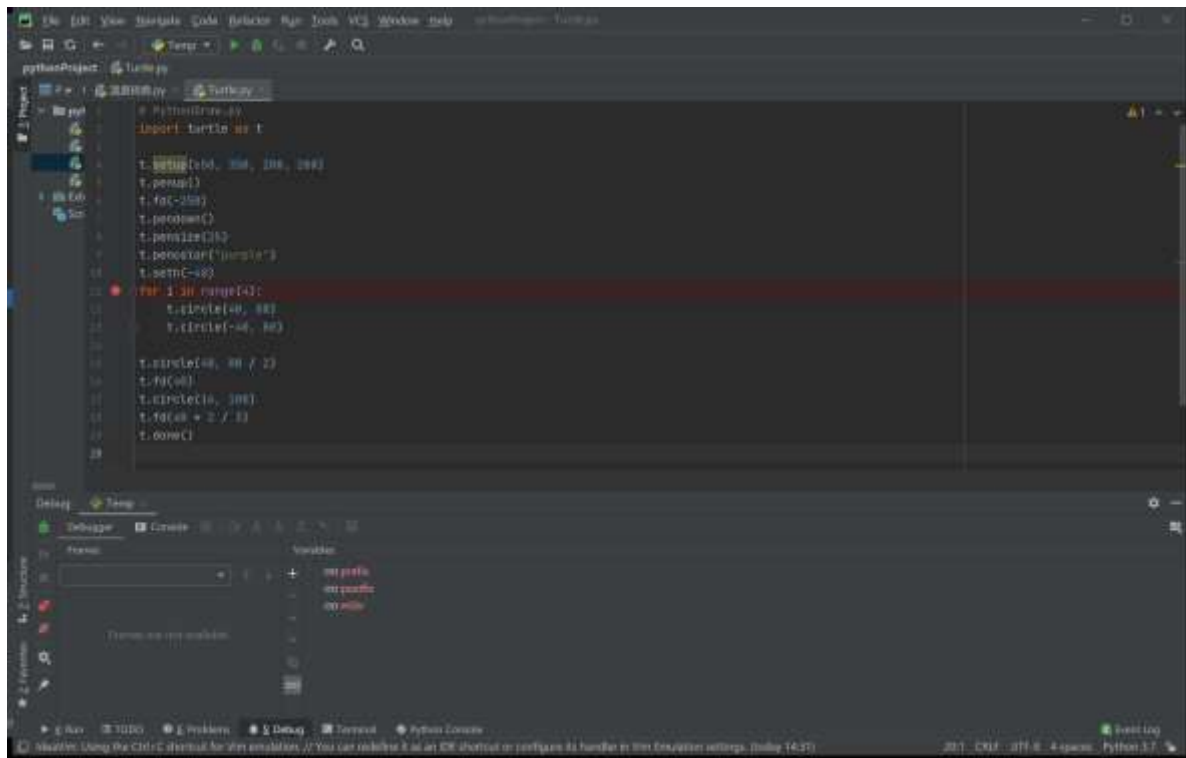
```
import turtle as t
```

```
t.setup(650 , 350, 200, 200)  
t.penup()  
t.fd(-250)  
t.pendown()  
t.pensize(25)  
t.pencolor("purple")  
t.seth(-40)  
for i in range(4):  
    t.circle(40,80)  
    t.circle(-40, 80)  
  
t.circle(40, 80/2)  
t.fd(40)  
t.circle(16,180)  
t.fd(40 * 2/3)  
t.done()
```

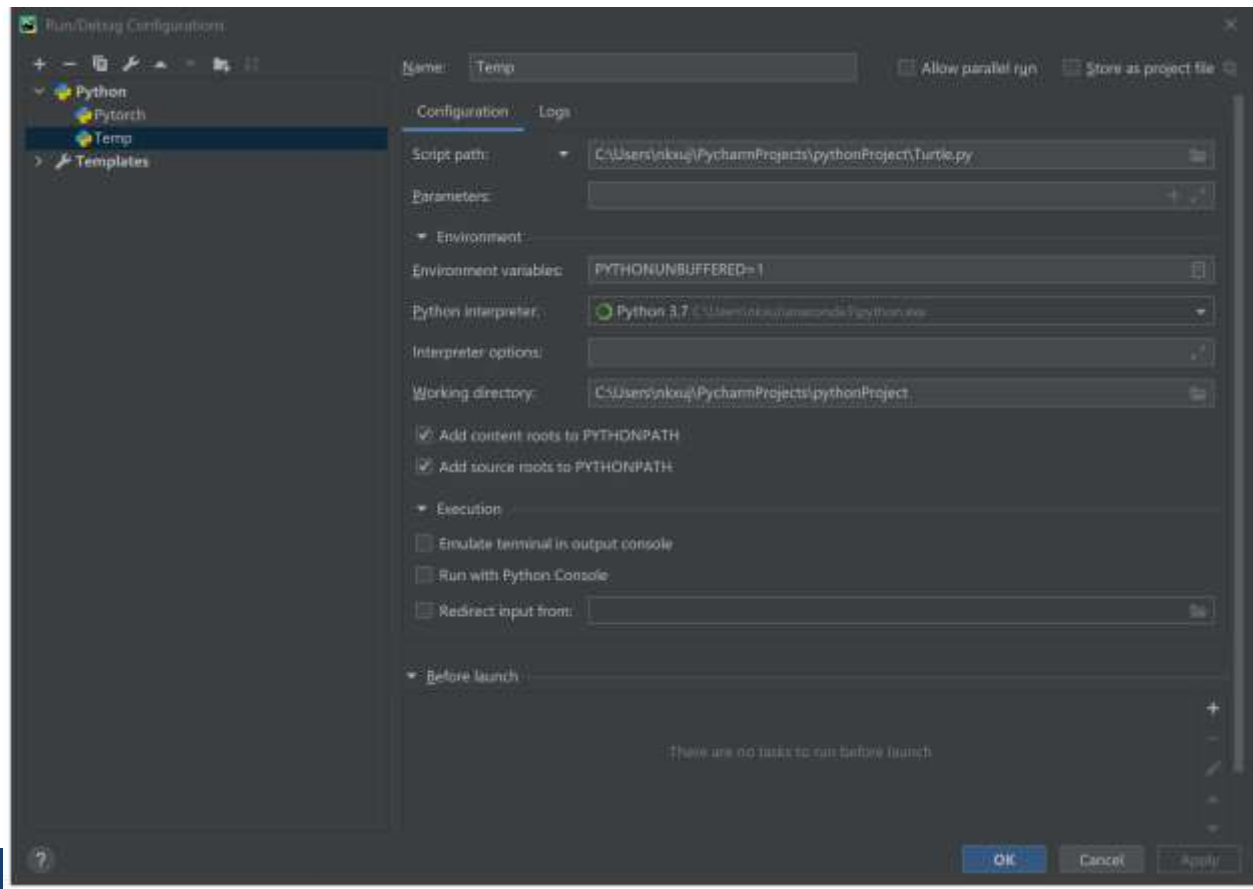


代码执行过程分析：在PyCharm中设置断点

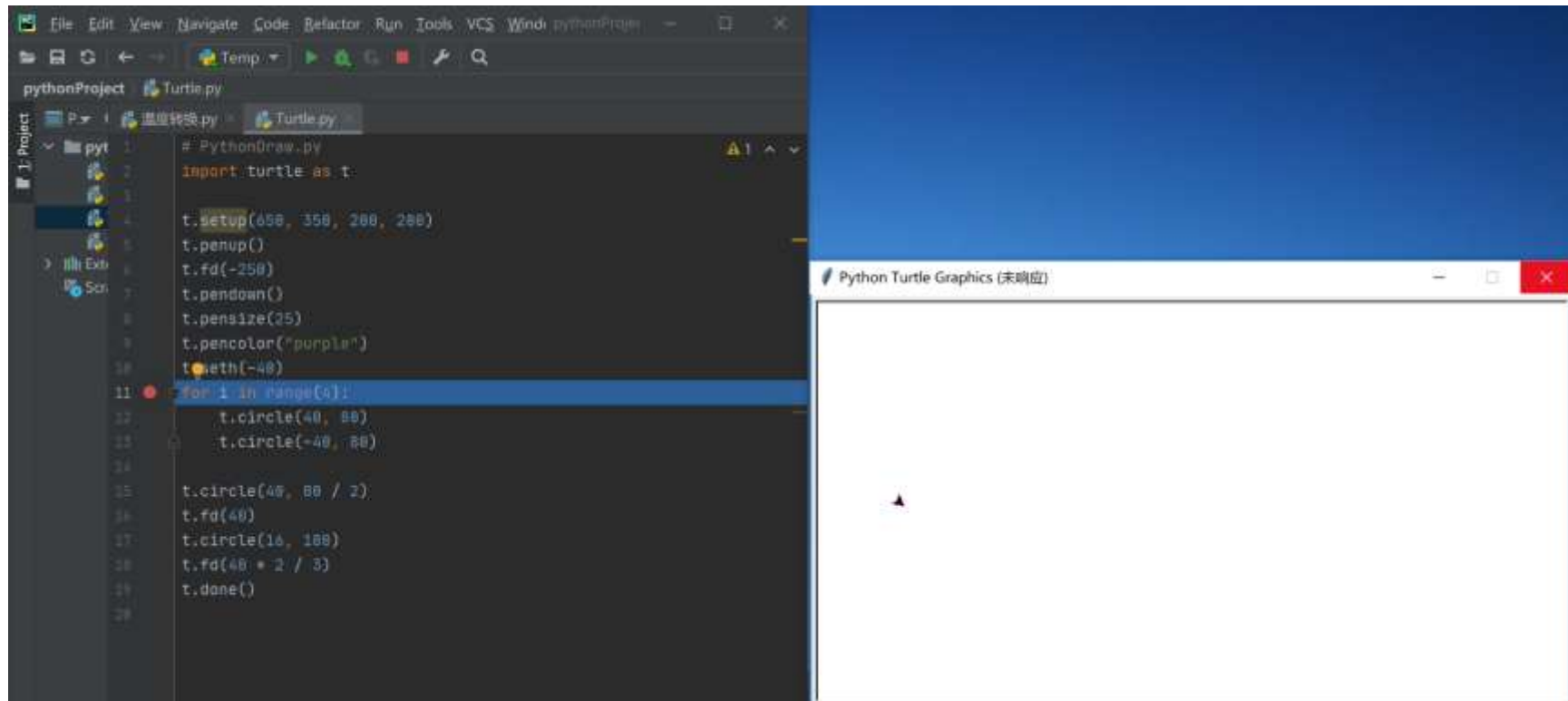
- 鼠标双击红点的位置
 - 可以设置多个
- 点击绿色小虫子运行
- 单步执行
 - 点击相应图标
 - F9: Resume
 - F8: Step over
 - F7: Step into
 - ...



在Debug Config窗口中设置Script Path



代码执行过程分析：画笔初始状态





代码执行过程分析：执行第一次循环

The image shows a Python IDE window with a file named `Turtle.py` open. The code is as follows:

```
# PythonDraw.py
import turtle as t

t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)

for i in range(4):
    t.circle(40, 80)

t.circle(40, 80 / 2)
t.fd(40)
t.circle(10, 180)
t.fd(40 * 2 / 3)
t.done()
```

The IDE is running the code, and the `for` loop is currently executing the first iteration (`i = 0`). The `t.circle(40, 80)` command has been executed, resulting in a purple arc being drawn on the canvas. The `Python Turtle Graphics` window shows the canvas with the arc.



```
File Edit View Navigate Code Refactor Run Tools VCS Window pythonProject
Temp
pythonProject Turtle.py
pyt PythonDraw.py
import turtle as t
t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4): i: #
    t.circle(40, 80)
    t.circle(-40, 80)
t.circle(40, 80 / 2)
t.fd(40)
t.circle(10, 180)
t.fd(40 * 2 / 3)
t.done()
```

Python Turtle Graphics

A purple wavy line drawing on a white background, resembling a stylized 'S' or a wave. The drawing is composed of several connected circular arcs, with the first two being larger and the subsequent ones being smaller.



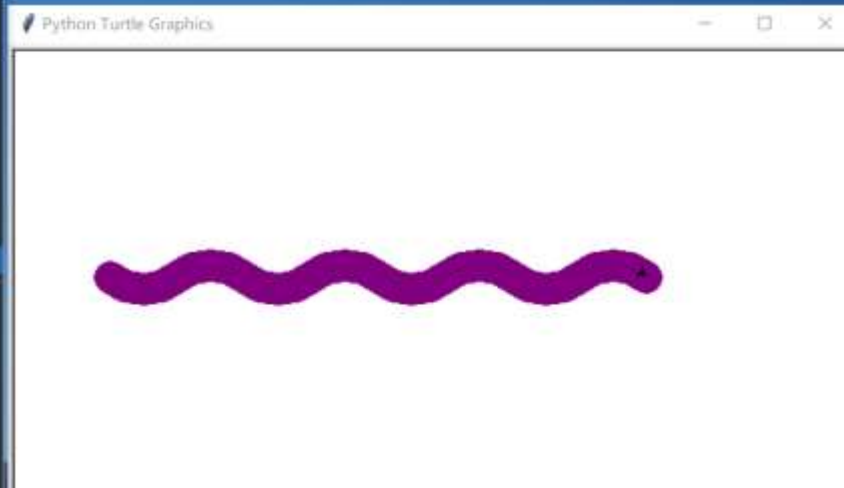
```
File Edit View Navigate Code Refactor Run Tools VCS: Wind pythonProject
Temp
pythonProject Turtle.py
pyt
1 E: PythonDraw.py
2 import turtle as t
3
4 t.setup(650, 350, 200, 200)
5 t.penup()
6 t.fd(-250)
7 t.pendown()
8 t.pensize(20)
9 t.pencolor("purple")
10 t.seth(-40)
11 for i in range(4):
12     t.circle(40, 80)
13     t.circle(-40, 80)
14
15 t.circle(40, 80 / 2)
16 t.fd(40)
17 t.circle(10, 180)
18 t.fd(40 * 2 / 3)
19 t.done()
20
```

Python Turtle Graphics

A purple wavy line is drawn on a white canvas. The line starts at the left edge, moves right, then curves up and right, then curves down and right, and finally curves up and right again, ending at the right edge. The line is thick and has a smooth, wavy appearance.

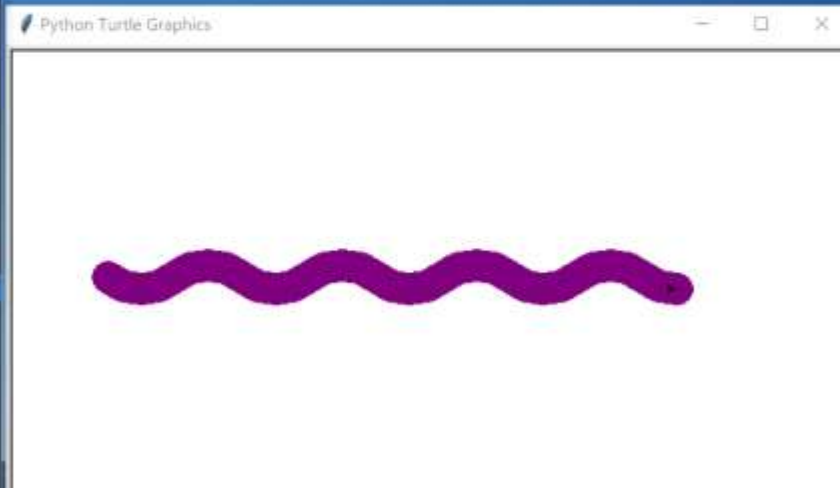


```
File Edit View Navigate Code Refactor Run Tools VCS Window pythonProject
Temp
pythonProject Turtle.py
pyt
1 # PythonDraw.py
2 import turtle as t
3
4 t.setup(650, 350, 200, 200)
5 t.penup()
6 t.fd(-250)
7 t.pendown()
8 t.pensize(25)
9 t.pencolor("purple")
10 t.seth(-40)
11 for i in range(4):
12     t.circle(40, 80)
13     t.circle(-40, 80)
14
15 t.circle(40, 80 / 3)
16 t.fd(40)
17 t.circle(16, 180)
18 t.fd(40 * 2 / 3)
19 t.done()
20
```





```
File Edit View Navigate Code Refactor Run Tools VCS Window pythonProject
Temp
pythonProject Turtle.py
pyt
1 # PythonDraw.py
2 import turtle as t
3
4 t.setup(650, 350, 200, 200)
5 t.penup()
6 t.fd(-250)
7 t.pendown()
8 t.pensize(20)
9 t.pencolor("purple")
10 t.seth(-40)
11 for i in range(4):
12     t.circle(40, 80)
13     t.circle(-40, 80)
14
15 t.circle(40, 80 / 2)
16 t.fd(40)
17 t.circle(10, 180)
18 t.fd(40 * 2 / 3)
19 t.done()
20
```



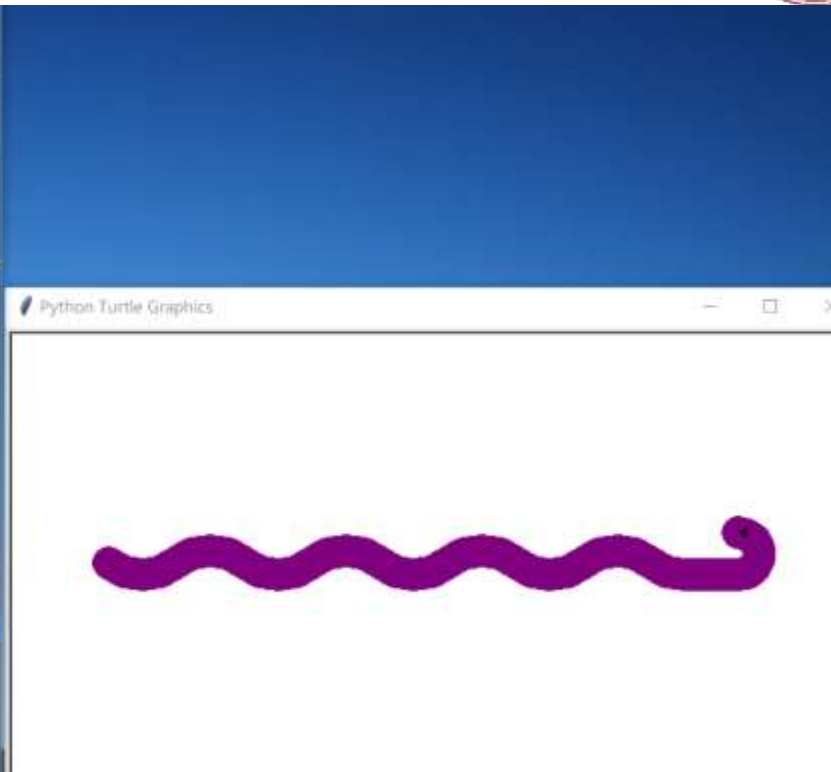


```
File Edit View Navigate Code Refactor Run Tools VCS Wind pythonProjec
Temp
pythonProject Turtle.py
pyt # PythonDraw.py
import turtle as t
t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40, 80)
    t.circle(-40, 80)
t.circle(40, 80 / 2)
t.fd(40)
t.circle(15, 100)
t.fd(40 * 2 / 3)
t.done()
```



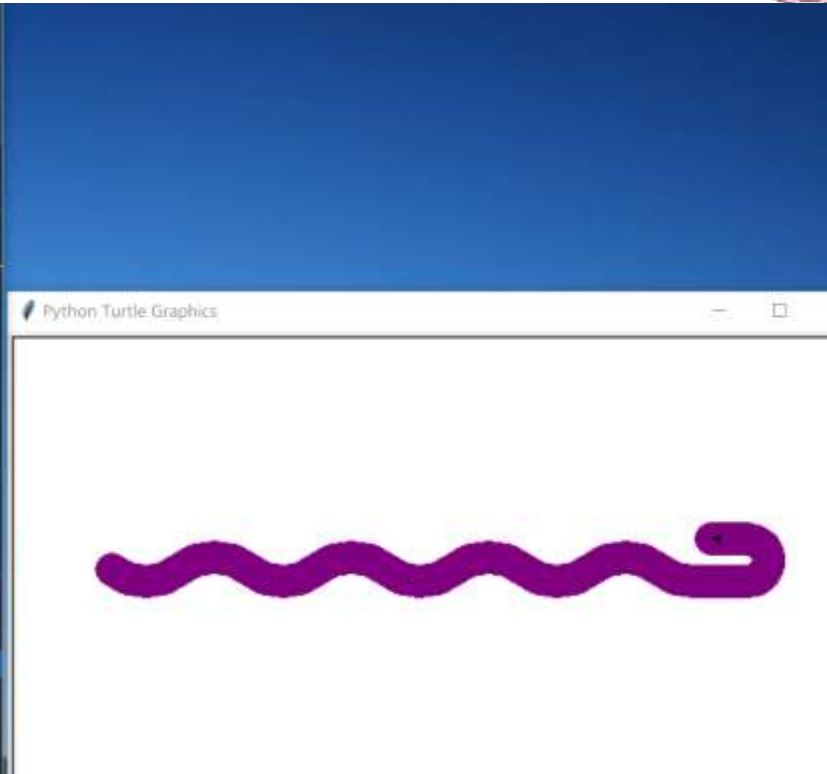


```
File Edit View Navigate Code Refactor Run Tools VCS Window pythonProject
Temp
pythonProject Turtle.py
pyt
import turtle as t
t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40, 80)
    t.circle(-40, 80)
t.circle(40, 80 / 2)
t.fd(40)
t.circle(16, 180)
t.fd(40 * 2 / 3)
t.done()
```



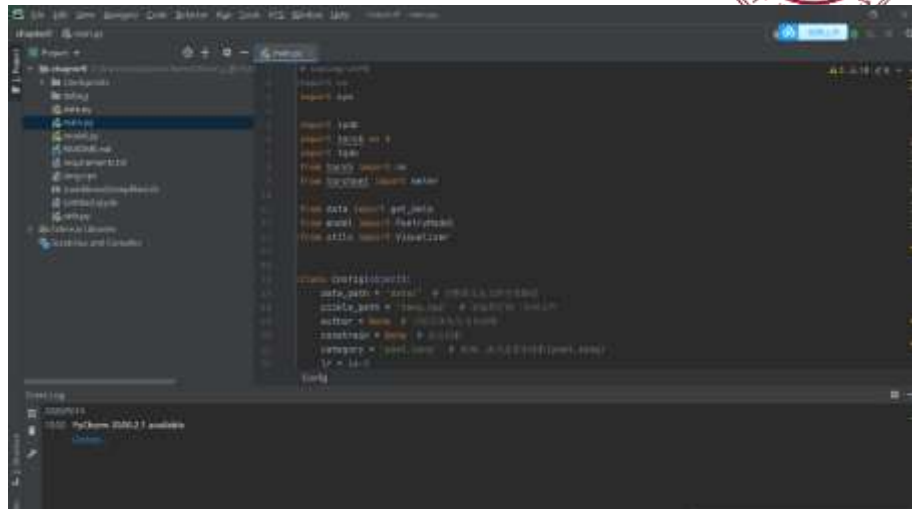


```
File Edit View Navigate Code Refactor Run Tools VCS Window pythonProject
Temp
pythonProject Turtle.py
pyt PythonDraw.py
import turtle as t
t.setup(650, 350, 200, 200)
t.penup()
t.fd(-250)
t.pendown()
t.pensize(25)
t.pencolor("purple")
t.seth(-40)
for i in range(4):
    t.circle(40, 80)
    t.circle(-40, 80)
t.circle(40, 80 / 2)
t.fd(40)
t.circle(10, 180)
t.fd(40 + 2 / 3)
t.done()
```



最后：关于IDE

- Jupiter Notebook适合学习，缺点也很明显
 - 无法单步执行跟踪程序执行中状态
 - 无法管理比较大的工程（多文件、多人协作）
 -
- Integrated Development Environment (IDE)
 - 高级编程者必备
 - PyCharm是一个比较流行的Python IDE
<https://www.jetbrains.com/pycharm/download/>
 - 其他IDE: eclipse、VSCode
 - 没有优劣之分



课后练习



- 修改绘制蟒蛇程序的参数，观察修改的效果
- 用turtle绘制正方形螺旋线
 - 用一个IDE编写
 - 如果程序运行出错
 - 仔细阅读出错信息，分析并修改你的程序
 - 如果执行效果与期待有差异
 - 通过单步执行观察每一行程序的执行效果是否符合你的设计，找出分析差异，修改你的程序



谢谢！